Storage and File Structure

Oleh:
Ir. Hira Laksmiwati Soemitro, M.Sc
Hartanto Tantriawan, S.Kom., M.Kom
Arief Ichwani, S.Kom., M.Cs

Program Studi Teknik Informatika
Institut Teknologi Sumatera
2026

Pokok Bahasan

1. Review
2. Storage dan Struktur File

Review - Arsitektur Sistem Basis Data

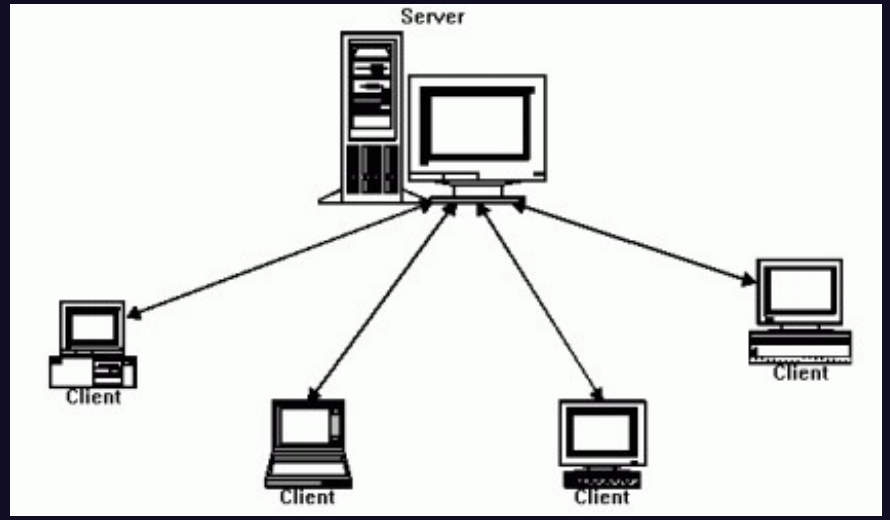
- Arsitektur sistem dipengaruhi oleh keberadaan basis data tersebut, dimana ia dijalankan.
- Tipe arsitektur sistem:
  - Arsitektur Terpusat
  - Arsitektur Client-Server
  - Arsitektur Terdistribusi

Review - Arsitektur terpusat

- Sistem berjalan pada sistem tunggal, tidak ada interaksi dengan sistem komputer lain.
- Berdasarkan unit yang digunakan, sistem terpusat dibagi 2:
  - Pengguna tunggal
  - Banyak pengguna, terdiri dari unit-unit terminal. Komputer mahal.

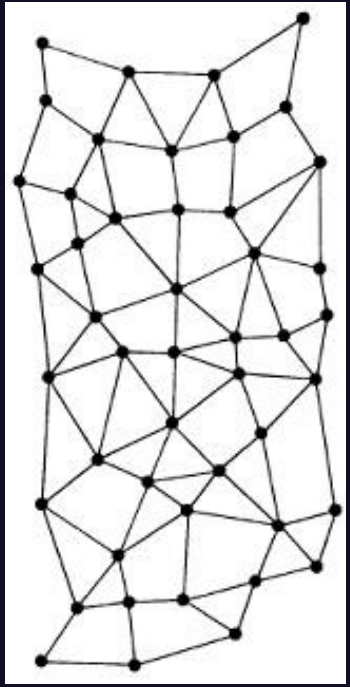
Review - Arsitektur Client Server

- Harga PC murah, PC menggantikan terminal pada arsitektur terpusat.
- Pemisahan pengolah transaksi (server) dan pengguna awam (client).



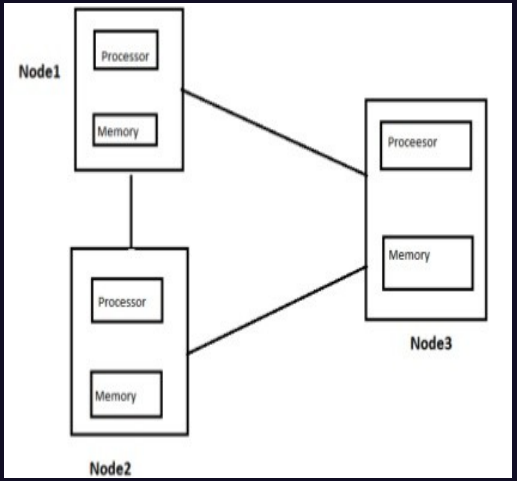
Review - Arsitektur Sistem Terdistribusi

- Basis data disimpan pada beberapa komputer. Seringkali disebut dengan istilah *node* atau *site*.
- Seringkali terpisah secara geografis.
- Berkomunikasi jaringan berkecepatan tinggi.



Review - Arsitektur Sistem Terdistribusi (cont)

- Basis data memiliki dua jenis transaksi:
  - Transaksi lokal
  - Transaksi global
- Contoh kasus:
  Sebuah bank memiliki 4 buah cabang provinsi. Nasabah ingin mengirimkan uang ke nasabah lain yang ada di Lampung. Apabila transaksi ini dimulai di Lampung, maka transaksi ini termasuk transaksi lokal. Apabila transaksinya dimulai dari luar provinsi Lampung, maka hal ini disebut transaksi global karena rekening diakses pada dua lokasi berbeda.



Review – Komponen Basis Data

- Skema basis data
- Tabel
- Field atau kolom
- Record atau baris
- Kunci
- Relasi
- Tipe data

Review – Skema Basis Data

Sekelompok objek dalam basis data yang saling berhubungan. Objek yang dihubungkan memiliki relasi satu sama lain. Model skema:

- Model internal
  Sering disebut dengan model fisik, berurusan dengan penyimpanan fisik basis data serta akses ke data seperti melalui penyimpanan data dalam tabel dan penggunaan indeks untuk mempercepat akses data. Model internal memisahkan kebutuhan fisik hardware dan sistem operasi dari model data
- Model konseptual
  Sering disebut dengan model logis, model basis data dasar yg berurusan dengan struktur organisasi yang digunakan untuk menentukan struktur basis data seperti tabel dan batasan.
- Model eksternal
  Sering disebut sebagai antarmuka aplikasi, berurusan dengan metode yang digunakan pengguna agar dapat mengakses skema seperti melalui penggunaan form input data.

Review – Tabel

- Unit penyimpan fisik data. Ketika mengakses basis data, pengguna mengacu tabel untuk data yang diinginkan.
- Terdapat banyak tabel pada satu basis data, satu sama lain dihubungkan dengan sebuah relasi.
- Macam tabel:
  - **Tabel data**, menyimpan data pada suatu basis data
  - **Join tabel**, tabel yang digunakan untuk membentuk relasi antara dua tabel
  - **Subset tabel**, mengandung sebagian data dari suatu tabel data
  - **Tabel validasi**, seringkali disebut sebagai tabel kode, digunakan untuk membuat valid data yang dimasukkan pada tabel lain.

Review – Kolom

- Disebut juga sebagai field. Dapat diartikan sebagai **ketegori informasi** yang terdapat dalam tabel.
- Kolom bagi tabel seperti atribut pada entitas. Ketika terjadi konversi, entitas menjadi tabel dan atribut menjadi kolom.
- Kolom mempunyai tipe. Tipe data menentukan tipe nilai yang dapat mengisi sebuah kolom.

Review – Baris

- Seringkali disebut dengan istilah *record*.
- Kumpulan semua kolom dalam sebuah tabel yang menggambarkan sebuah kejadian tunggal (data tunggal).
- Contoh:
  Apabila terdapat 15.000 data buku, maka akan terdapat 15.000 baris data dalam tabel buku. Jumlah baris pada tabel akan berubah ketika judul buku bertambah atau berkurang.

Review – Tipe data

- Menentukan jenis data yang dapat disimpan dalam sebuah kolom tabel.
- Umumnya terdapat tiga buah tipe data:
  - Alfanumerik
    Menyimpan karakter, angka, karakter khusus, dan hampir semua kombinasi. Nilai numerik yang disimpan dalam tipe alfanumerik akan diperlakukan sebagai karakter bukan angka.
  - Numerik
    Menyimpan nilai-nilai numerik.
  - Tanggal dan waktu
    Menyimpan nilai tanggal, waktu, atau kombinasi keduanya.

Review – Model Basis Data

- Model basis data file (*flat file database model*)
- Model basis data hierarki (*hierarchical database model*)
- Model basis data jaringan (*network database model*)
- Model basis data relasional (*relational database model*)
- Model basis data berorientasi objek (*object oriented database model*)

Review – Model Basis Data Flat File

- Tersusun dalam satu atau lebih file dan disimpan dalam bentuk teks.
- Informasi dalam file teks disimpan sebagai field.
- Field dapat memiliki panjang yang tetap atau berubah-ubah (dipisahkan oleh tanda tertentu).

Review – Model Basis Data Flat File
(panjang field tetap)

File buku.txt

```
1234    Wahana Komputer       Administrasi Sistem di Linux
5678    Dony Ariyus           Kamus Hacker
4321    Komarudin             Mendulang Dollar dari Google
8765    Agfianto Putra        Belajar Bahasa Assembly
3456    Abdul Kadir           Bahasa Pemrograman Perl
```

- Basis data berisi 3 field dengan panjang tetap. Yaitu nomor buku, nama pengarang, judul buku.
- Kolom #1 s.d kolom #4 adalah data nomor buku.
- Kolom #8 s.d kolom #23 adalah data penulis.
- Dst...

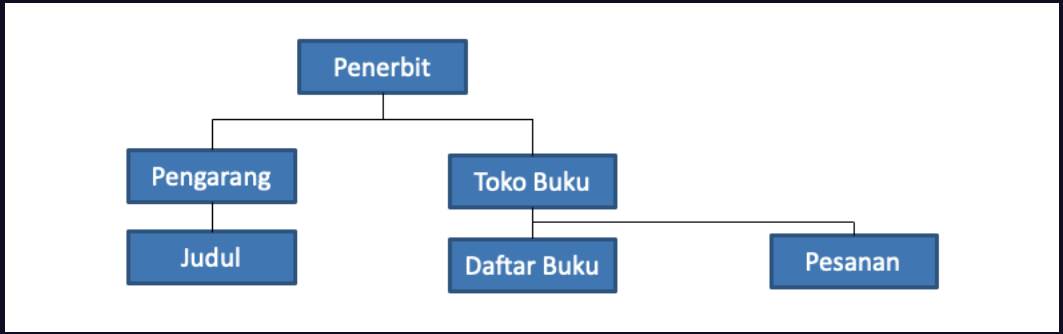
Review – Model Basis Data Flat File
(panjang field berubah)

File buku.txt

```
1234:Wahana Komputer:Administrasi Sistem di Linux
5678:Dony Ariyus:Kamus Hacker
4321:Komarudin:Mendulang Dollar dari Google
8765:Agfianto Putra:Belajar Bahasa Assembly
3456:Abdul Kadir:Bahasa Pemrograman Perl
```

Review – Model Basis Data hierarchical

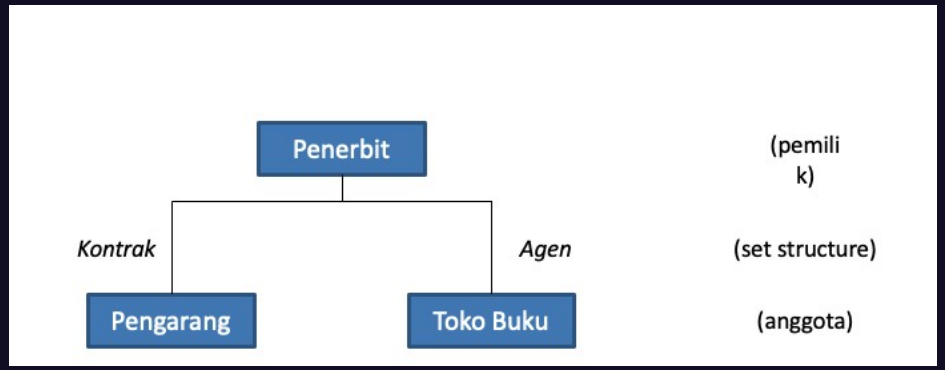
- Memiliki tingkatan diatas flat file.
- Landasan nya pada konsep hubungan orang tua dan anak.
- Tabel akar atau tabel orang tua berapada pada puncak dan menunjuk pada tabel anak yang mengandung data yang berhubungan.



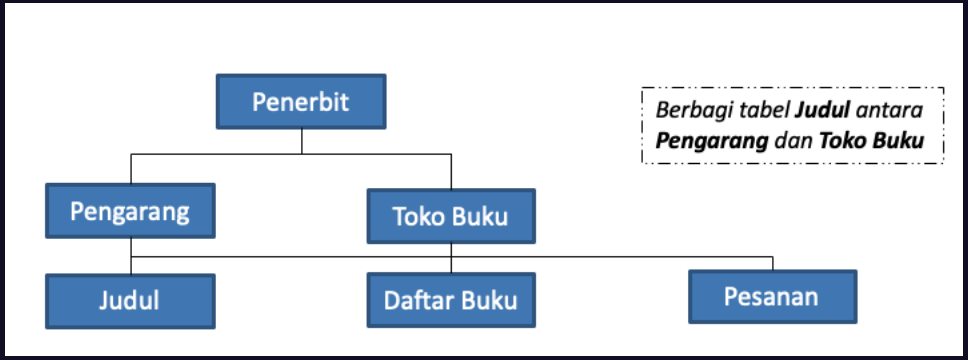
- Penerbit memiliki banyak pengarang, dan toko buku sebagai agen penjualan.
- Tabel orang tua dapat memiliki banyak anak, tetapi anak hanya memiliki satu orang tua.
- Penerbit adalah tabel orang tua dari Pengarang dan Toko Buku
- dst...

Review – Model Basis Data jaringan

- Penyempurnaan dari basis data hirarki dengan kemampuan berbagi hubungan dengan tabel anak.
- Tabel anak dapat memiliki beberapa tabel orang tua. Pengguna dapat mengakses data dari sembarang tabel.
- Hubungan antar tabel disebut sebagai *set structure*, dimana satu tabel adalah pemilik dan tabel lain adalah anggota. Mewakili hubungan one-many antar tabel.
- Program aplikasi yang mengakses basis data jaringan menggunakan set structure untuk mengarahkan kebagian tabel lain. Jika set structure berubah, program aplikasi juga harus berubah.

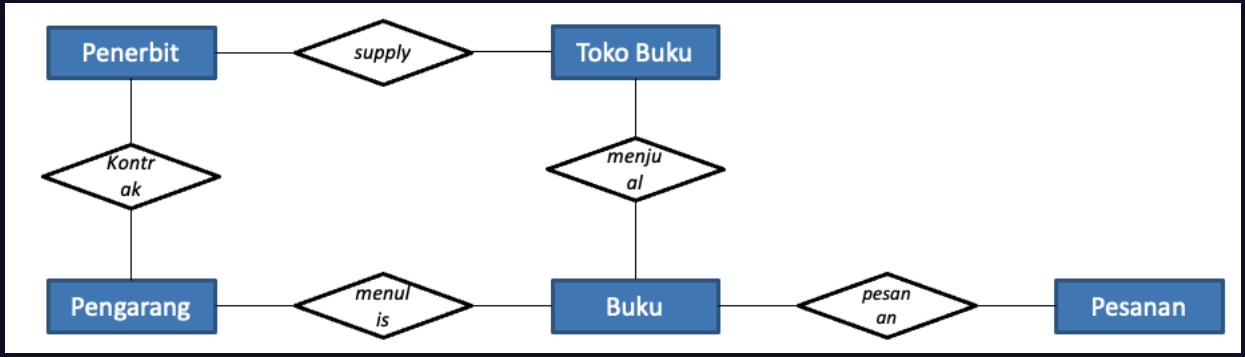


Review – Model Basis Data jaringan



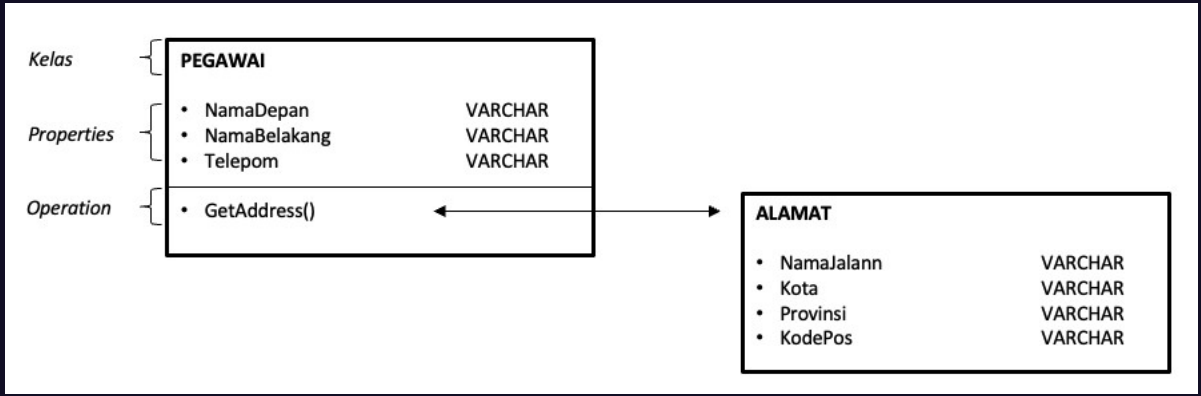
- Tabel anak dimiliki oleh lebih dari satu tabel orang tua. Tabel **Judul** dimiliki oleh tabel **Pengarang** dan **Daftar Buku**.
- Tabel judul disimpan pada satu tabel saja. Sehingga mengurangi pengulangan data.

Review – Model basis data relasional



- Model yang umum atau populer digunakan saat ini. Data menjadi semakin mudah diatur.
- Model relasional tidak memiliki tabel akar, meskipun demikian hubungan antara orang tua dan anak masih dimungkinkan.
- Hubungan antar tabel: one-one, one-many, many-many.
- Tabel dihubungkan satu dengan tabel lain melalui kolom yang disebut kunci (*key*).

Review – Model basis data berorientasi objek



- Model baru dalam pengembangan basis data.
- Basis data dimana data dapat ditentukan, disimpan, dan diakses menggunakan pendekatan pemrograman berorientasi objek.
- Menggunakan pendekatan berorientasi objek untuk menentukan struktur basis data.

Storage and File Structure

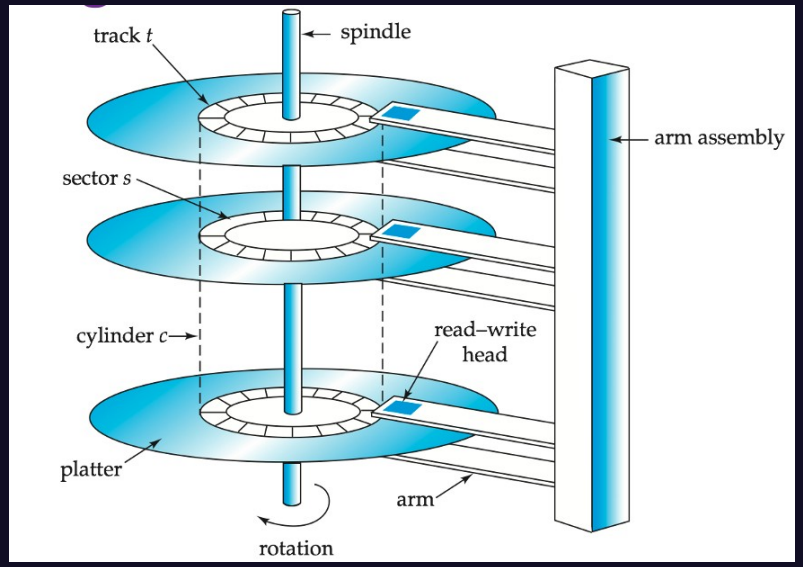
- Mekanisme kerja magnetic disk
- Peran Buffer Manager
- Cara akses secondary storage
- Organisasi file

Mekanisme Kerja Magnetic Disk

Storage Hierarchy

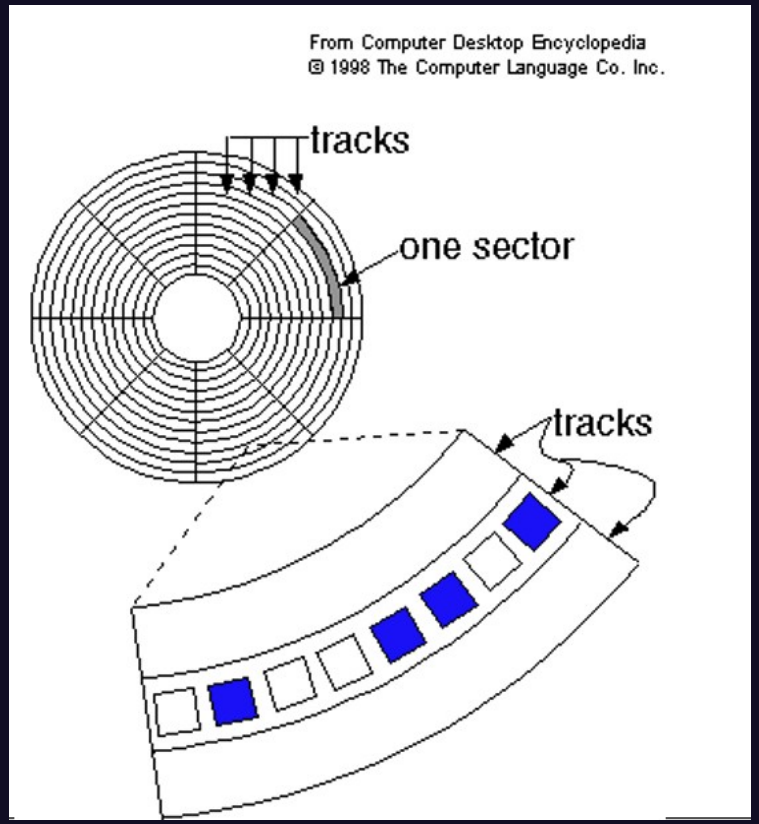
- **primary storage**: Fastest media but volatile (cache, main memory).
- **secondary storage**: next level in hierarchy, non-volatile, moderately fast access time
  - also called **on-line storage**
  - E.g. flash memory, magnetic disks
- **tertiary storage**: lowest level in hierarchy, non-volatile, slow access time
  - also called **off-line storage**
  - E.g. magnetic tape, optical storage

Magnetic Hard Disk Mechanism



- **NOTE**: Diagram is schematic, and simplifies the structure of actual disk drives

Magnetic Hard Disk Tracks and Sectors



- Surface of platter divided into circular **tracks**
  - Over 50K-100K tracks per platter on typical hard disks
- Each track is divided into **sectors**.
  - A sector is the smallest unit of data that can be read or written.
  - Sector size typically 512 bytes
  - Typical sectors per track: 500 to 1000 (on inner tracks) to 1000 to 2000 (on outer tracks)

Magnetic Disks

Read-write head

- Komponen dalam hard disk drive (HDD) atau perangkat penyimpanan magnetik lainnya yang bertanggung jawab untuk membaca (retrieving) dan menulis (storing) data ke dalam platter.
- bekerja menggunakan prinsip elektromagnetik dan biasanya melayang di atas permukaan platter dengan jarak yang sangat kecil untuk menghindari kontak fisik yang dapat merusak disk.

Arm (Lengan Aktuator)

- Bagian dari mekanisme hard disk yang menopang **Read-write head** dan memindahkannya ke berbagai lokasi pada platter. Arm ini dikontrol oleh aktuator untuk memastikan **Read-write head** dapat mencapai trek yang diinginkan dengan presisi tinggi.

Platter (Piringan Disk)

- Komponen utama dalam hard disk yang berbentuk cakram datar dan berputar dengan kecepatan tinggi. Platter biasanya terbuat dari material non-magnetik seperti aluminium atau kaca dan dilapisi dengan bahan magnetik yang memungkinkan penyimpanan data.

Magnetic Disks (Cont)

Disk controller

- Sebuah sirkuit elektronik yang bertindak sebagai perantara antara sistem komputer dan hard disk drive. Disk controller bertanggung jawab untuk menerjemahkan perintah dari sistem operasi ke dalam instruksi yang dapat dijalankan oleh disk, mengontrol gerakan kepala baca-tulis, dan menangani transfer data.

Cylinder

- Kumpulan dari track yang berada pada posisi yang sama di setiap platter dalam sebuah hard disk. Karena hard disk terdiri dari beberapa platter yang disusun secara vertikal, setiap track dengan nomor yang sama pada setiap platter membentuk satu silinder. Akses ke data sering kali dilakukan dalam satu silinder untuk meningkatkan efisiensi.

Sector

- Unit terkecil dari penyimpanan data dalam disk. Setiap platter dibagi menjadi beberapa sektor yang biasanya memiliki ukuran 512 byte atau 4096 byte dalam disk modern. Sektor adalah bagian dari trek dan digunakan sebagai unit baca/tulis data.

Magnetic Disks (Cont)

Spindle

- Komponen dalam hard disk yang menopang dan memutar platter pada kecepatan tinggi, biasanya antara 5.400 hingga 15.000 RPM (rotasi per menit).

Track

- Lingkaran konsentris pada permukaan platter tempat data disimpan secara magnetik. Setiap platter memiliki banyak trek, dan setiap trek terdiri dari beberapa sektor. Semakin tinggi kapasitas disk, semakin banyak trek yang tersedia.

Performance Measures of Disks

Access Time - Waktu yang dibutuhkan oleh hard disk untuk menemukan dan membaca atau menulis data pada platter. **Terdiri dari:**

- **Seek time** - Waktu yang dibutuhkan **Read-write head** untuk berpindah ke trek yang sesuai.
- **Rotational latency** – Waktu yang dibutuhkan sektor yang diinginkan untuk berada tepat di bawah **Read-write head**.
- **Data-transfer rate** – Waktu yang dibutuhkan untuk membaca atau menulis data dari sektor yang dipilih.

- Rumus menghitung **rotational latency**:

$$\text{Rotational Latency} = \frac{60}{2 \times \text{RPM}} \text{ (dalam milidetik)}$$

- Contoh untuk HDD dengan **7.200 RPM**:

$$\frac{60}{2 \times 7200} = 4.17 \text{ ms}$$

Perbandingan rata-rata transfer rate:

- **HDD SATA III (7200 RPM):** ~100-150 MB/s
- **SSD SATA III:** ~500-600 MB/s
- **SSD NVMe Gen 3:** ~1.5-3.5 GB/s
- **SSD NVMe Gen 4:** ~5-7 GB/s

Performance Measures (continue)

Mean time to failure (MTTF)

- MTTF adalah estimasi jumlah waktu operasional sebelum sebuah perangkat (seperti hard disk) mengalami kegagalan. Biasanya diukur dalam **jam** dan digunakan untuk menentukan keandalan perangkat keras.

- Contoh MTTF untuk beberapa perangkat:
  - HDD konvensional: **500.000 – 1.500.000 jam**
  - SSD modern: **1.500.000 – 2.000.000 jam**
- MTTF tidak menjamin umur perangkat secara pasti, tetapi memberikan gambaran statistik tentang seberapa andal perangkat sebelum kemungkinan terjadi kegagalan.
- Performa MTTF akan menurun sesuai usia perangkat.

Peran Buffer Manager

Storage Access

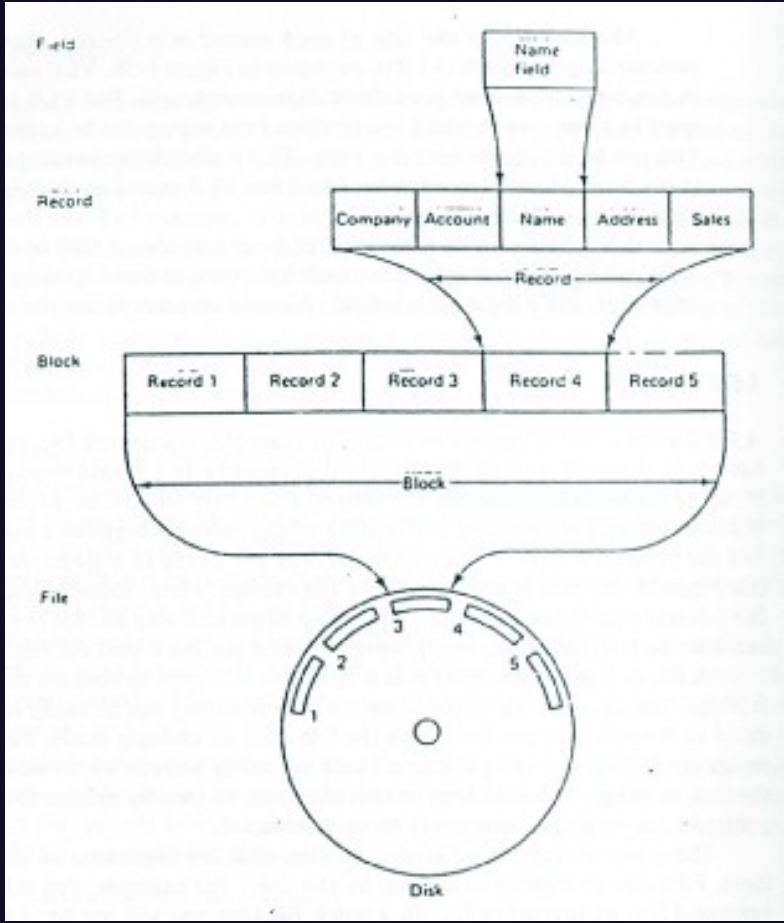
- Dalam sistem database, data disimpan dalam bentuk file yang dibagi menjadi **blok-blok** dengan ukuran tetap. **Blok (block)** adalah unit dasar penyimpanan yang digunakan untuk menyimpan sejumlah record atau data. Ukuran blok bervariasi tergantung pada sistem file dan database, tetapi umumnya berkisar antara **512 byte hingga 8 KB atau lebih**.
- Sistem database berusaha **meminimalisir** jumlah transfer blok antara disk dan memori. **Akses ke memori utama lebih cepat** dibandingkan dengan akses ke disk, sehingga sistem database mencoba mengurangi jumlah operasi baca/tulis ke disk. Semakin sedikit transfer blok antara disk dan memori, semakin cepat kinerja sistem database.
- **Buffer** adalah area di RAM tempat sistem database menyimpan **blok-blok yang baru saja diambil dari disk**. Ketika aplikasi membutuhkan data, sistem akan **memeriksa buffer terlebih dahulu** sebelum mengakses disk. Jika data sudah ada di buffer, maka akses disk tidak diperlukan (**cache hit**). Jika tidak ada, sistem harus mengambilnya dari disk (**cache miss**).
- **Buffer manager** adalah subsistem yang bertanggung jawab dalam mengalokasikan ruang buffer di memori utama. **Buffer manager** bertugas mengatur bagaimana blok disimpan, diperbarui, dan dihapus dari buffer.

Strategi untuk mengurangi transfer Block

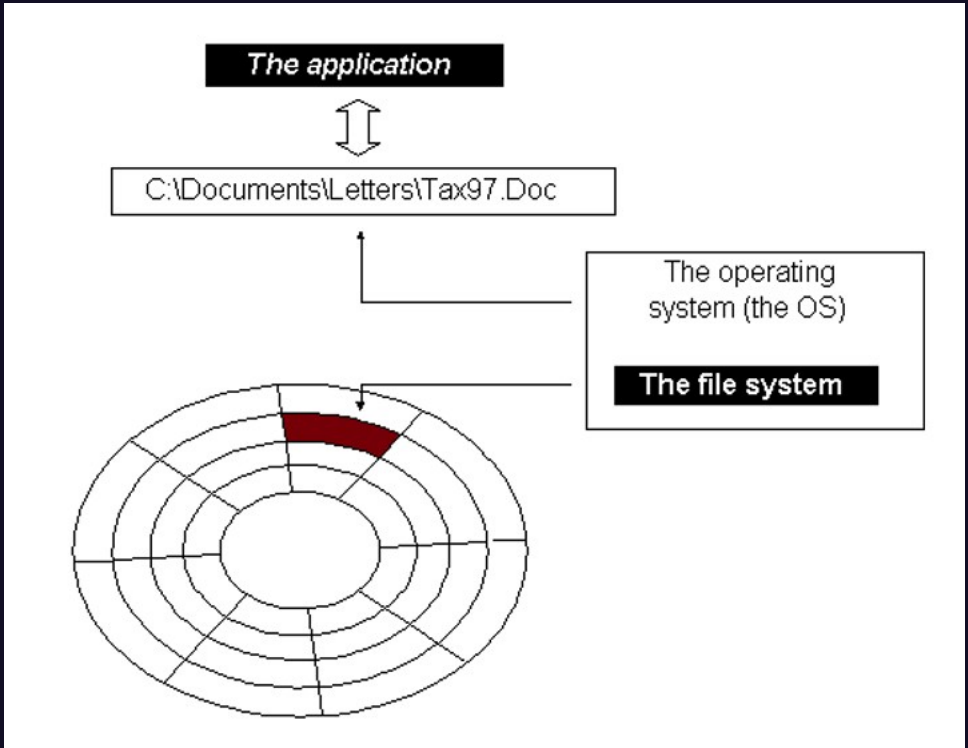
Ada empat cara untuk mengatasi nya:

- **Caching**: Menyimpan blok yang sering diakses dalam memori utama agar tidak perlu membaca ulang dari disk.
- **Buffering**: Menggunakan buffer di memori utama untuk menyimpan sementara blok yang sedang digunakan.
- **Indexing**: Menggunakan indeks untuk mencari data dengan lebih sedikit akses disk.
- **Clustered Storage**: Menyimpan data yang sering
- digunakan bersama dalam blok yang berdekatan untuk mengurangi jumlah transfer blok.

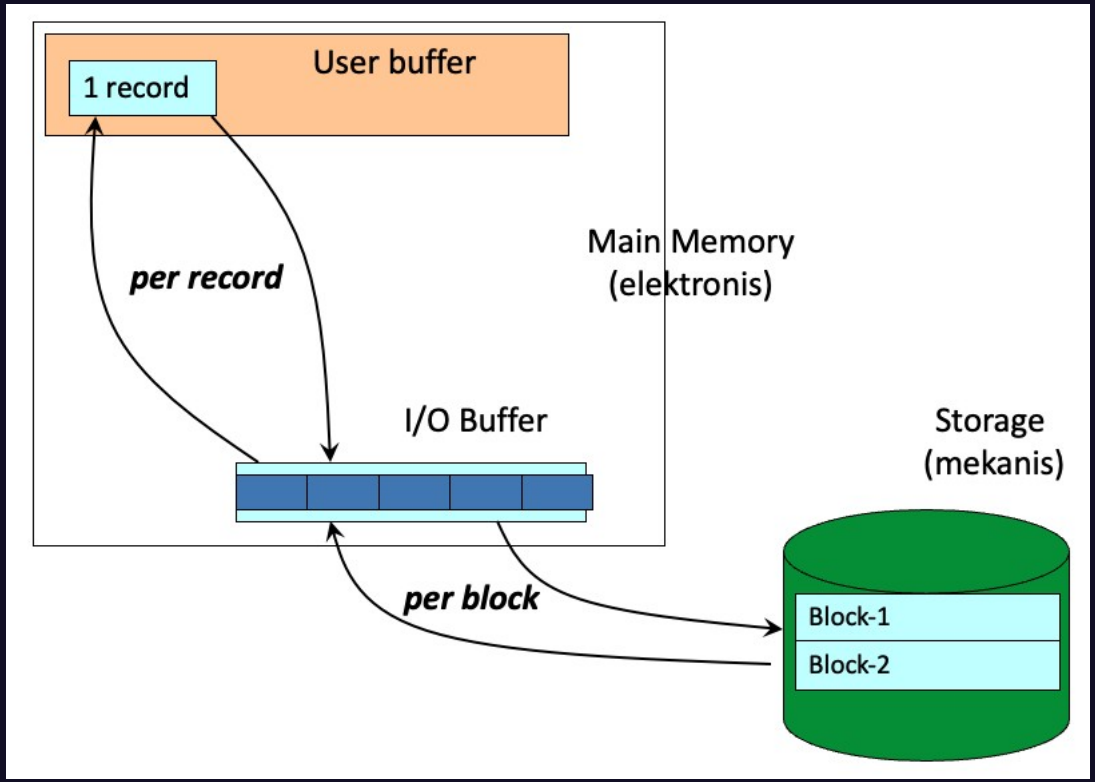
Fields, Records, and Files



Disk-Block Transfer



Record Access Mechanism



Optimasi Akses Blok Disk menggunakan Nonvolatile Write Buffer

Secara umum, kecepatan penulisan data ke piringan disk (HDD) jauh lebih lambat dibandingkan kecepatan prosesor. Untuk mengatasi hambatan (*bottleneck*) ini, digunakan teknik **Nonvolatile Write Buffer**.

Mekanisme Nonvolatile Write Buffer

- **Penyimpanan Instan ke Buffer**: Sistem mempercepat proses penulisan dengan menyimpan blok data ke dalam *buffer* RAM non-volatil secara seketika, alih-alih langsung menuliskannya ke piringan fisik disk.
- **Komponen Hardware**: RAM non-volatil ini biasanya berupa RAM yang didukung oleh cadangan baterai (*battery-backed up RAM*) atau menggunakan memori *flash*.
- **Ketahanan Data (Data Persistence)**: Keunggulan utamanya adalah keamanan data; meskipun terjadi kegagalan daya (mati listrik), data tetap tersimpan aman di dalam *buffer* dan akan segera ditulis ke disk saat daya kembali normal.
- **Manajemen Kontroler**: Kontroler disk akan menjalankan proses penulisan dari *buffer* ke disk fisik hanya pada saat disk sedang tidak melayani permintaan lain (idle) atau jika ada permintaan yang sudah mengantre terlalu lama.

Buffer Manager

Buffer Manager adalah komponen system yang bertanggung jawab menangani permintaan blok data antara memori utama dan penyimpanan disk.
Prosedur permintaan block

- **Cek Ketersediaan**: Jika blok yang diminta sudah ada di *buffer*, manajer akan langsung memberikan alamat blok tersebut di memori utama.
- **Alokasi Blok Baru**: Jika blok tidak tersedia di buffer, manajer buffer akan:
  - Menyiapkan ruang di *buffer* (jika penuh, akan dilakukan penggantian/*replacement* blok lama).
  - **Write-Back**: Blok lama yang diganti akan ditulis kembali ke disk hanya jika data tersebut telah dimodifikasi (mengalami perubahan) sejak terakhir kali diambil.
  - **Read & Fetch**: Membaca blok baru dari disk ke dalam *buffer* dan memberikan alamat memorinya kepada peminta.

Buffer-Replacement Policies

- **Pinned block** – memory block that is not allowed to be written back to disk.
- **Toss-immediate** strategy – frees the space occupied by a block as soon as the final tuple of that block has been processed
- **Most recently used (MRU) strategy** – system must pin the block currently being processed. After the final tuple of that block has been processed, the block is unpinned, and it becomes the most recently used block.
- Buffer manager can use statistical information regarding the probability that a request will reference a particular relation
  - E.g., the data dictionary is frequently accessed. Heuristic: keep data-dictionary blocks in main memory buffer
- Buffer managers also support **forced output** of blocks for the purpose of recovery

Cara Akses Secondary Storage

Optimization of Disk Block Access

File organization – Mengoptimalkan waktu akses blok dengan mengorganisasi blok sesuai dengan pola akses data.

- Contoh: Database yang sering mengakses **data pelanggan dan transaksi bersama-sama** akan menyimpan keduanya dalam blok yang berdekatan agar akses lebih cepat.
- Menyimpan informasi yang saling terkait pada silinder yang sama atau berdekatan.

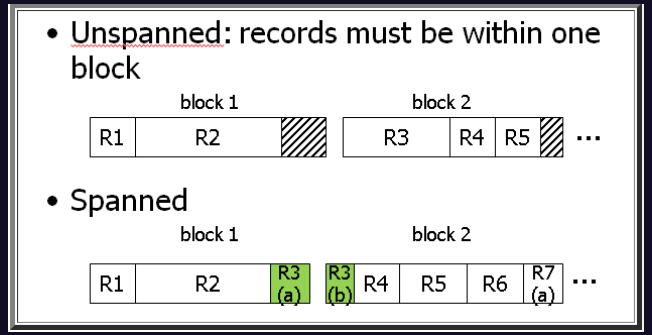
File Organization, Record Organization, And Storage Access

File Organization

- Basis data disimpan sebagai kumpulan file. Setiap file merupakan urutan dari rekaman (*records*), dan setiap rekaman terdiri dari urutan bidang (*fields*).
- **Pendekatan Rekaman Ukuran Tetap (*Fixed-Length Records*)**: Salah satu pendekatan awal adalah dengan mengasumsikan bahwa ukuran rekaman bersifat tetap
- **Keseragaman Tipe Rekaman**: Dalam metode ini, setiap file hanya berisi rekaman dari satu tipe tertentu saja.
- **Pemisahan Relasi**: File yang berbeda digunakan untuk menyimpan relasi (tabel) yang berbeda pula.

Records and Blocking

- Proses penyesuaian atau penempatan rekaman (*records*) ke dalam blok-blok penyimpanan disebut dengan ***blocking***.
- ***Blocking factor***, yang disimbolkan dengan ***Bfr***, adalah jumlah rekaman yang diharapkan atau dapat ditampung di dalam satu blok.
- roses *blocking* dapat dibedakan menjadi dua jenis, yaitu ***spanned*** (terentang) atau ***unspanned*** (tidak terentang)..



Fixed-Lenth Records

- Simple approach:
  - Rekaman ke-i disimpan mulai dari ***byte* n * (i - 1)**, di mana n adalah ukuran dari setiap rekaman..
  - Akses rekaman menjadi sangat sederhana, namun terdapat kemungkinan rekaman terpecah di antara dua blok (*cross blocks*).
  - Sistem dapat dimodifikasi untuk tidak mengizinkan rekaman melintasi batas blok.

  - Untuk metode *unspanned fixed blocking*, kapasitas blok dihitung dengan rumus:

    $Bfr = \lfloor B/R \rfloor$ where

    B is the block size and

    R is the record size

- Deletion of record *i:*

  alternatives*:*

  1. move records $i + 1, \ldots, n$ to $i, \ldots, n - 1$
  2. move record n to i
  3. do not move records, but link all free records on a *free list*

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

1. Deleting record 3 and compacting

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

2. Deleting record 3 and moving last record

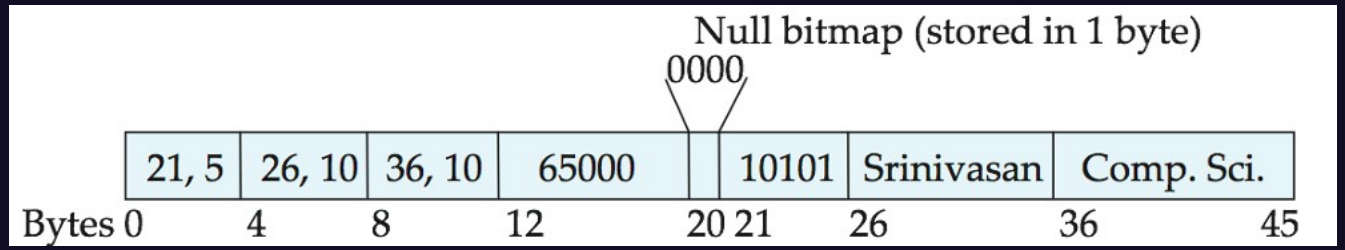
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

4. Contoh

- Misal :
  1 block dapat menyimpan 250 byte, jika 1 record panjangnya 100 byte maka BLOCKING SBB:

Variable-Length Records

- Variable-length records arise in database systems in several ways:
  - Storage of multiple record types in a file.
  - Record types that allow variable lengths for one or more fields such as strings (**varchar**)
  - Record types that allow repeating fields (used in some older data models).
- Attributes are stored in order
- Variable length attributes represented by fixed size (offset, length), with actual data stored after all fixed length attributes
- Null values represented by null-value bitmap

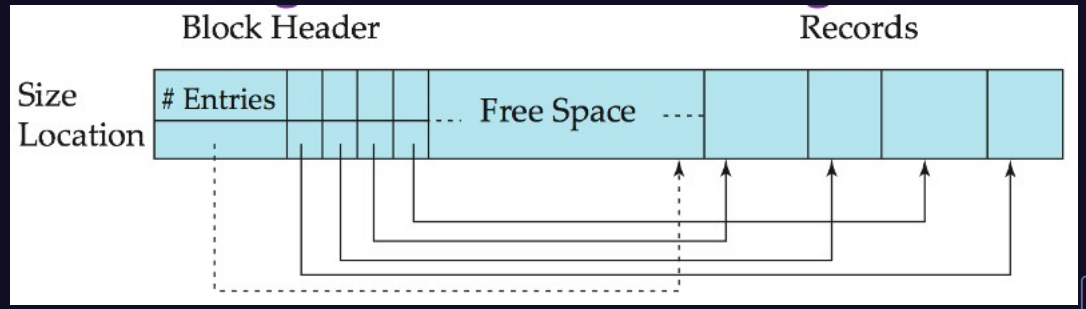


5. Contoh Variable Length Records

- Misal : 1 blok dapat memuat 100 byte.
  - PANJANG RECORD 1 = 80 BYTE
  - PANJANG RECORD 2 = 40 BYTE
  - PANJANG RECORD 3 = 50 BYTE

BLOK - 1		BLOK -2 DST….		
REC -1	REC-2	REC – 2	REC – 3	KOSONG
80 byte	20 BYTE	20 BYTE	50 BYTE	

Variable-Length Records : Slotted Page Structure



- **Slotted page**
  header contains:
  - number of record entries
  - end of free space in the block
  - location and size of each record
- Records can be moved around within a page to keep them contiguous with no empty space between them; entry in the header must be updated.
- Pointers should not point directly to record — instead they should point to the entry for the record in header.

File Performance Parameters

1. R: The amount of storage required for a record
2. T_F: The time needed to fetch an arbitrary record from the file
3. T_N: The time needed to get the next record within the file
4. T_I: The time required to update the file by inserting a record
5. T_U: The time required to update the file by changing a record
6. T_X: The time needed for exhaustive reading of the entire file
7. T_O: The time needed for reorganization of the file

Record Size

- Beside the data, a record may also stores **descriptive** and **access information**.
- Descriptive information is advantageous when storing heterogeneous data.

Fetch A Record

To be able to use data from a file, a record containing the data has to be read into the memory.

- **Fetch** is an *associative* retrieval of a data element based on a key value.
- Fetching a record consists of **2 steps**:
  - Locating the position of the record
  - The actual reading
- In general, the records of a file cannot be directly located on the basis of a subscript value or record number
  - Therefore, a simple address computation cannot be performed

Get the next Record

Information is mainly generated by relating one fact with another; this implies getting the next record according to some criterion.

- **Get-Next** is a retrieval using a *structural dependency*.
- A successor record can be obtained most rapidly when related data is kept together; that is, when the locality of these data is strong.
    - When the records are not physically ordered according to the criterion, Get-Next is time consuming.

Insert A Record

Most applications require regular insertion of new data records into their files to remain up to date.

- **Inserting a record comprises of**:
  - Locating the block where the record will be inserted
  - Reading the block
  - Rewriting the changed block
- When a record has to be put in a specific place within the file, other records may have to be shifted or modified.
- Insertions to the end of the file, Append, are easier to handle.
  - If the address of the last block of the file is known, the performance of this operation can be improved.

Update A Record

All changes of data do not require insertion of a new record.

- When data within a stored record must be changed, the new, updated record is created.
  - The old data and the changes are merged to create a new record,
  - The new record is inserted into the position of the old record within the block, and
  - The block is rewritten into the file.
- If the record has grown in size, it may not be possible to use the old position → **delete** the old record and **insert** a new one.
- Deleting a record is not performed immediately, but instead the record to be deleted is marked with an indicator that the record is now invalid → the delete is now converted to an update.

Organisasi File

Reorganize the File

Reorganization removes deleted and invalidated records, reclaims the space for new data, and restores clustering.

- It is especially important for file organizations which create tombstones for deletion and updates.
- Its frequency depends on the type of file organization used but varies greatly with the application.
- It has many similarities with “garbage collection”.

Organization of Records in File

- **Heap** – a record can be placed anywhere in the file where there is space
- **Sequential** – store records in sequential order, based on the value of the search key of each record
- **Hashing** – a hash function computed on some attribute of each record; the result specifies in which block of the file the record should be placed
- Records of each relation may be stored in a separate file. In a **multitable clustering file organization** records of several different relations can be stored in the same file
  - Motivation: store related records on the same block to minimize I/O

Sequential File Organization

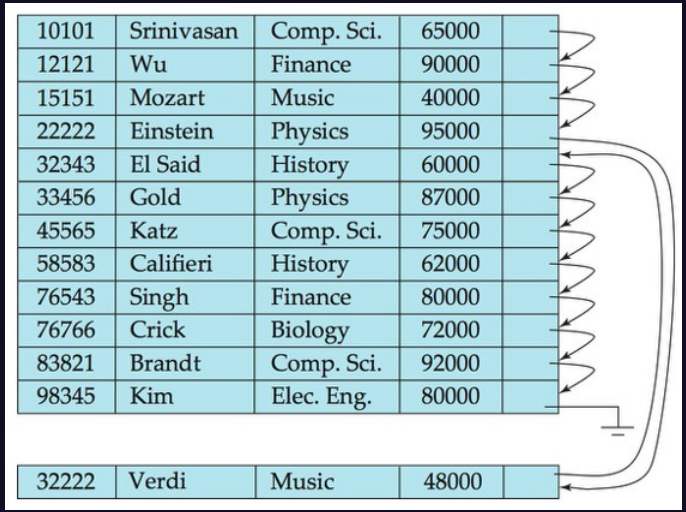
- Suitable for applications that require sequential processing of the entire file
- The records in the file are ordered by a **search-key**

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

Pointer

Sequential File Organization (continue)

- Deletion – use pointer chains
- Insertion – locate the position where the record is to be inserted
  - if there is free space insert there
  - if no free space, insert the record in an **overflow block**
  - In either case, pointer chain must be updated
- Need to reorganize the file from time to time to restore sequential order



Multitable Clustering File Organization

Store several relations in one file using a **multitable clustering** file organization

department

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Physics	Watson	70000

instructor

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

multitable clustering of *department* and *instructor*

Comp. Sci.	Taylor	100000	← *batas*
45564	Katz	75000	
10101	Srinivasan	65000	
83821	Brandt	92000	
Physics	Watson	70000	← *batas*
33456	Gold	87000	

Multitable Clustering File Organization (Cont)

- **good** for queries involving the join of *department* and *instructor*, and for queries involving one single department and its instructors
- **bad** for queries involving only *department*
- results in variable size records
- Can add **pointer chains** to link records of a particular relation

Comp. Sci.	Taylor	100000
45564	Katz	75000
10101	Srinivasan	65000
83821	Brandt	92000
Physics	Watson	70000
33456	Gold	87000

Data Dictionary Storage

The **Data dictionary** (also called **system catalog**) stores **metadata**; that is, data about data, such as:

- Information about relations
  - names of relations
  - names, types and lengths of attributes of each relation
  - names and definitions of views
  - integrity constraints
- User and accounting information, including passwords
- Statistical and descriptive data
  - number of tuples in each relation
- Physical file organization information
  - How relation is stored (sequential/hash/...)
  - Physical location of relation
- Information about indices

Use of Index

- Indexing mechanisms used to speed up access to desired data.
  - E.g., author catalog in library
- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form

search-key	pointer

- Index files are typically much smaller than the original file
- Two basic kinds of indices:
  - **Ordered indices**: search keys are stored in sorted order
  - **Hash indices**: search keys are distributed uniformly across “buckets” using a “hash function”.

Index Evaluation Metrics

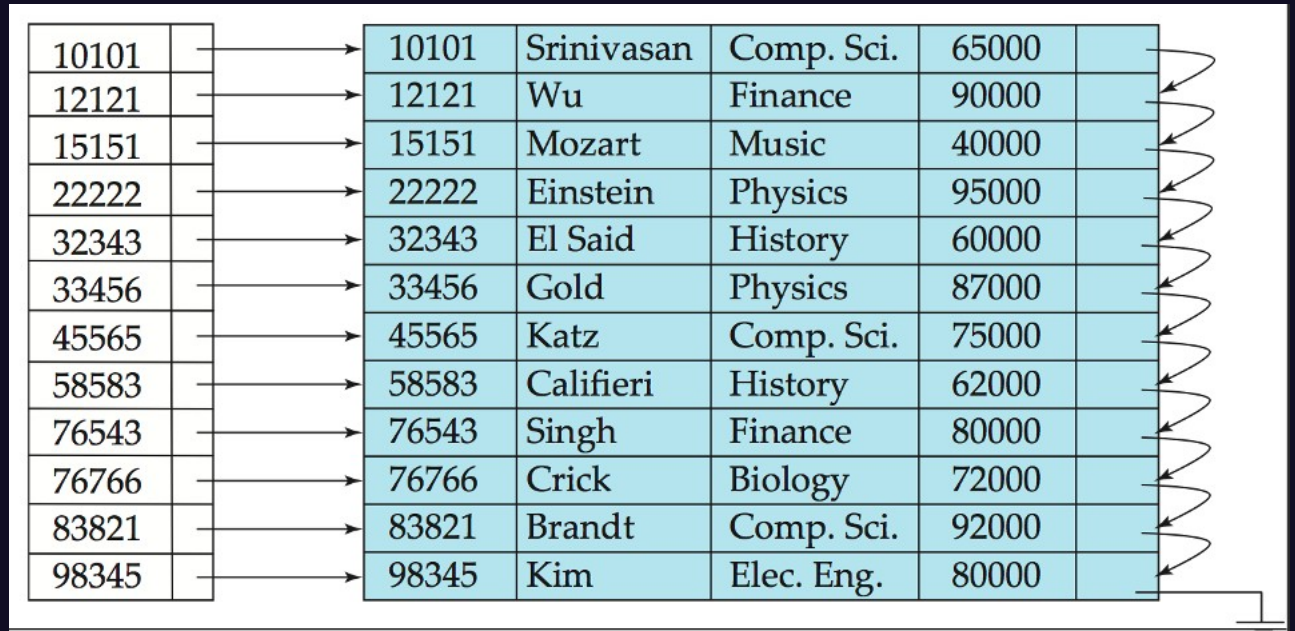
- Access types supported efficiently. E.g.,
  - Find records with a specified value in the attribute
  - or records with an attribute value falling in a specified range of values.
- Access time
- Insertion time
- Deletion time
- Space overhead

Ordered Index

- In an **ordered index**, index entries are stored sorted on the search key value. E.g., author catalog in library.
- **Primary index**: in a sequentially ordered file, the index whose search key specifies the sequential order of the file.
  - Also called **clustering index**
  - The search key of a primary index is usually but not necessarily the primary key.
- **Secondary index**: an index whose search key specifies an order different from the sequential order of the file. Also called non-clustering index.
- Index-sequential file: ordered sequential file with a primary index.

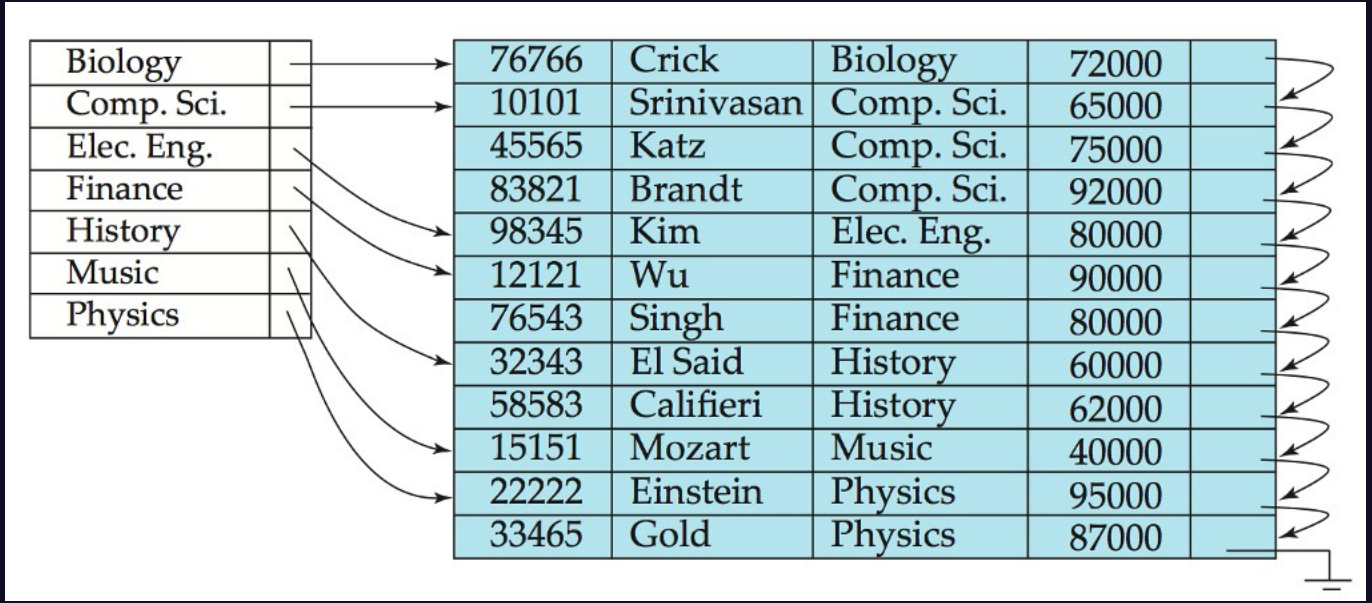
Dense Index File

- **Dense index** — Index record appears for every search-key value in the file.
- E.g. index on *ID* attribute of *instructor* relation



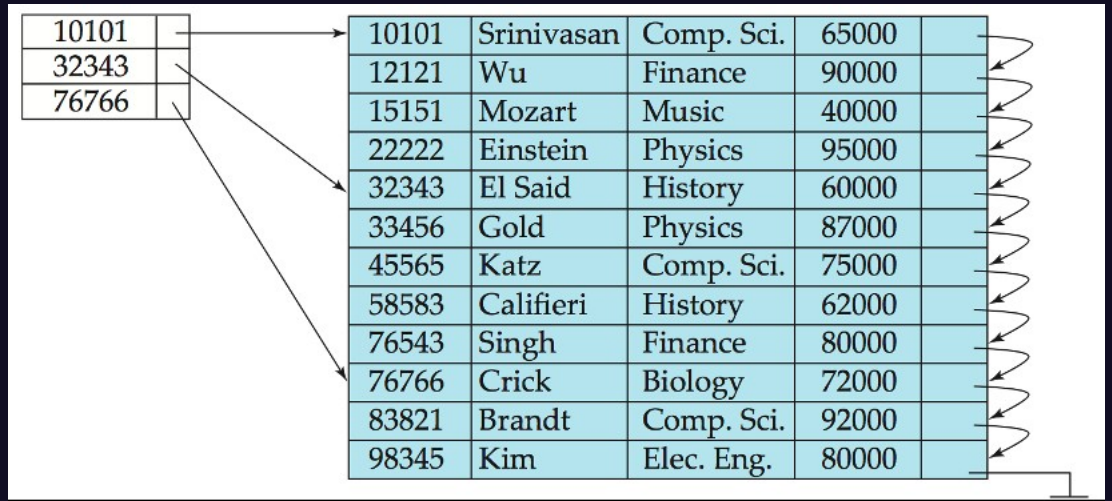
Dense Index File

Dense index on *dept_name*, with *instructor* file sorted on *dept_name*



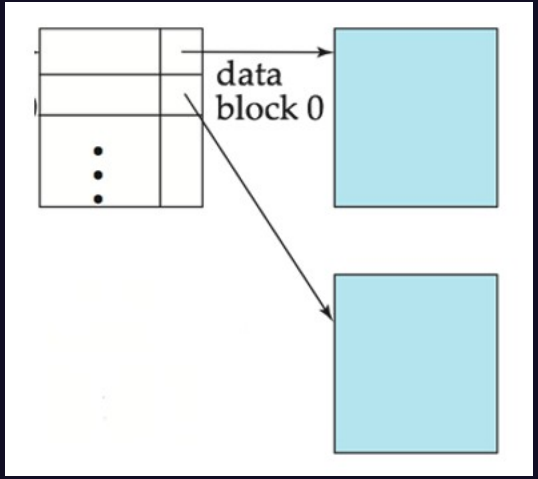
Sparse Index File

- **Sparse Index**: contains index records for only some search-key values.
  - Applicable when records are sequentially ordered on search-key
- To locate a record with search-key value *K* we:
  - Find index record with largest search-key value < *K*
  - Search file sequentially starting at the record to which the index record points



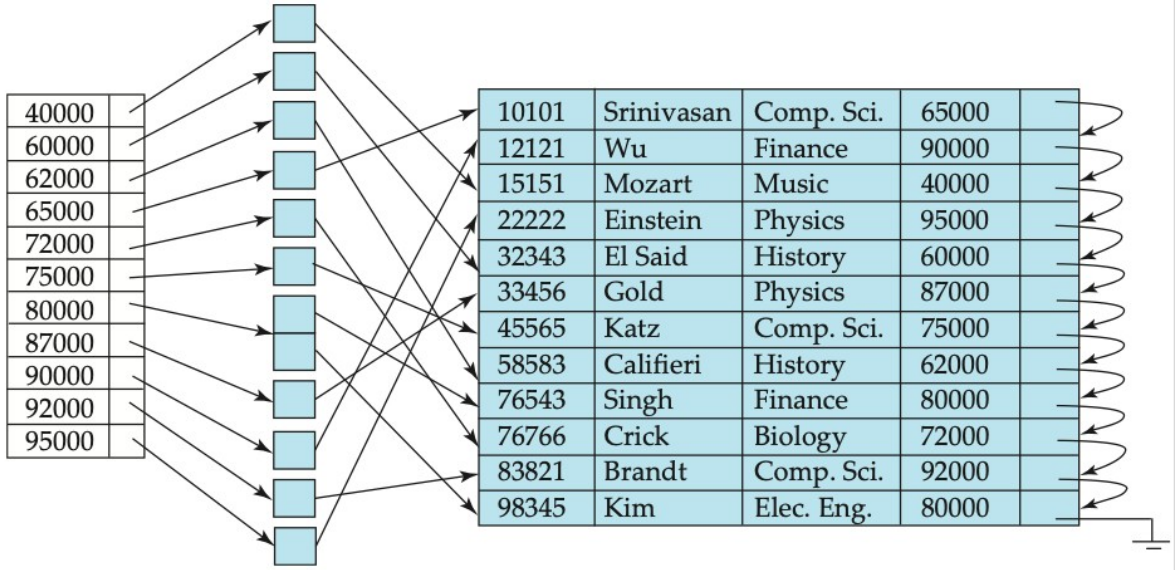
Sparse Index File (continue)

- Compared to dense indices:
  - Less space and less maintenance overhead for insertions and deletions.
  - Generally slower than dense index for locating records.
- **Good tradeoff**: sparse index with an index entry for every block in file, corresponding to least search-key value in the block.



Secondary Index (example)

- Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.
- Secondary indices have to be dense



Secondary index on *salary* field of *instructor*

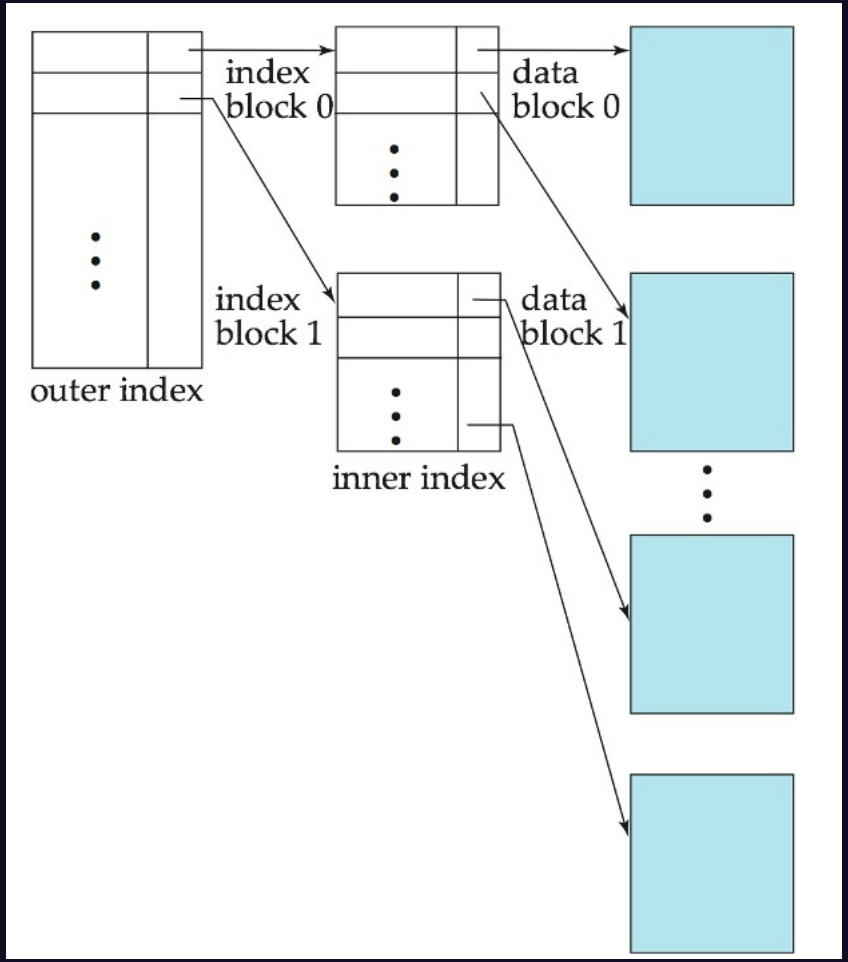
Primary and Secondary Indices

- Indices offer substantial benefits when searching for records.
- **BUT**: Updating indices imposes overhead on database modification --when a file is modified, every index on the file must be updated,
- Sequential scan using primary index is efficient, but a sequential scan using a secondary index is expensive
  - Each record access may fetch a new block from disk
  - Block fetch requires about 5 to 10 milliseconds, versus about 100 nanoseconds for memory access

Multilevel Index

- If primary index does not fit in memory, access becomes expensive.
- Solution: treat primary index kept on disk as a sequential file and construct a sparse index on it.
  - outer index – a sparse index of primary index
  - inner index – the primary index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- Indices at all levels must be updated on insertion or deletion from the file.

Multilevel Index



Index update (Deletion)

- If deleted record was the only record in the file with its particular search-key value, the search-key is deleted from the index also.
- **Single-level index entry deletion:**
  - **Dense indices** – deletion of search-key is similar to file record deletion.
  - **Sparse indices** –
    - if an entry for the search key exists in the index, it is deleted by replacing the entry in the index with the next search-key value in the file (in search-key order).
    - If the next search-key value already has an index entry, the entry is deleted instead of being replaced.

Index update (Insertion)

- **Single-level index insertion**:
  - Perform a lookup using the search-key value appearing in the record to be inserted.
  - **Dense indices** – if the search-key value does not appear in the index, insert it.
  - **Sparse indices** – if index stores an entry for each block of the file, no change needs to be made to the index unless a new block is created.
    - If a new block is created, the first search-key value appearing in the new block is inserted into the index.
- **Multilevel insertion and deletion:** algorithms are simple extensions of the single-level algorithms

Arigatou Gozaimasu